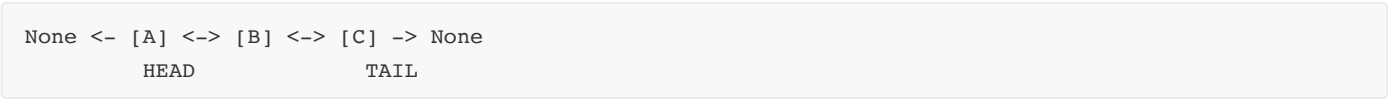


더블 링크드 리스트 함수 설명

1. 더블 링크드 리스트란?

더블 링크드 리스트(Doubly Linked List)는 각 노드가 이전 노드와 다음 노드를 모두 참조하는 자료구조이다.



각 노드는 다음 세 가지 정보를 가진다.

속성	설명
value	노드가 저장하는 실제 값
prev	이전 노드를 가리키는 참조
next	다음 노드를 가리키는 참조

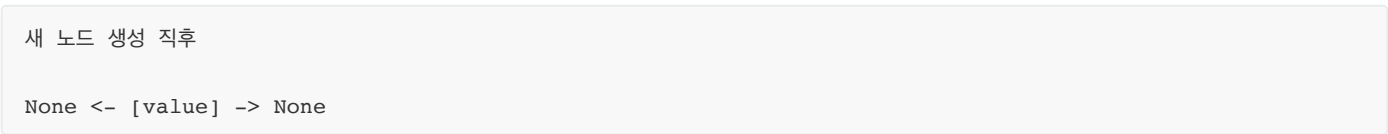
head는 첫 번째 노드를 가리키고, tail은 마지막 노드를 가리킨다.

2. Node 클래스

```
class Node:
    def __init__(self, value):
        self.value = value
        self.prev = None
        self.next = None
```

Node 클래스는 더블 링크드 리스트를 구성하는 노드 하나를 표현한다.

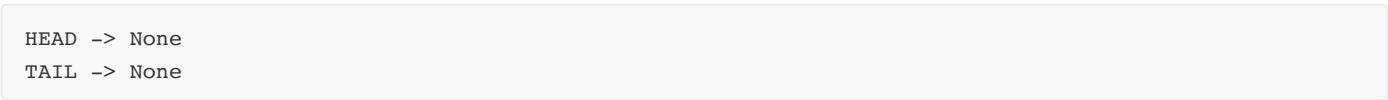
새 노드는 아직 다른 노드와 연결되지 않았으므로 prev와 next를 None으로 초기화한다.



3. DoubleLinkedList 생성자

```
def __init__(self):
    self.head = None
    self.tail = None
```

DoubleLinkedList 객체가 처음 만들어졌을 때는 저장된 노드가 없다.



- head는 첫 번째 노드를 가리킨다.
- tail은 마지막 노드를 가리킨다.

- 리스트가 비어 있으면 두 값 모두 `None` 이다.

4. append(value)

`append()` 는 더블 링크드 리스트의 맨 뒤에 새 노드를 추가한다.

```
linked_list.append("A")
linked_list.append("B")
linked_list.append("C")
```

결과:

```
None <- [A] <-> [B] <-> [C] -> None
      HEAD          TAIL
```

기존 마지막 노드와 새 노드를 양방향으로 연결해야 한다.

```
새 노드.prev = 기존 tail
기존 tail.next = 새 노드
tail = 새 노드
```

리스트가 비어 있다면 새 노드가 첫 번째이자 마지막 노드가 된다.

```
head = 새 노드
tail = 새 노드
```

`tail` 을 가지고 있으므로 맨 뒤 삽입은 $O(1)$ 에 처리할 수 있다.

5. prepend(value)

`prepend()` 는 더블 링크드 리스트의 맨 앞에 새 노드를 추가한다.

```
linked_list.prepend("A")
linked_list.prepend("B")
```

결과:

```
None <- [B] <-> [A] -> None
      HEAD    TAIL
```

새 노드와 기존 첫 번째 노드를 양방향으로 연결한다.

```
새 노드.next = 기존 head
기존 head.prev = 새 노드
head = 새 노드
```

리스트가 비어 있다면 `append()` 와 마찬가지로 `head` 와 `tail` 이 모두 새 노드를 가리킨다.

맨 앞 노드에 바로 접근할 수 있으므로 시간 복잡도는 $O(1)$ 이다.

6. insert(index, value)

`insert()` 는 원하는 인덱스에 새 노드를 삽입한다.

```
linked_list.append("A")
linked_list.append("C")
linked_list.insert(1, "B")
```

삽입 전:

```
[A] <-> [C]
```

삽입 후:

```
[A] <-> [B] <-> [C]
```

가운데에 노드를 삽입할 때는 네 개의 연결을 수정한다.

```
새 노드.prev = 이전 노드
새 노드.next = 현재 노드
이전 노드.next = 새 노드
현재 노드.prev = 새 노드
```

특수한 위치는 기존 메서드를 재사용한다.

조건	처리
<code>index < 0</code>	삽입 실패
<code>index == 0</code>	<code>prepend()</code> 호출
<code>index == length()</code>	<code>append()</code> 호출
중간 인덱스	해당 위치까지 이동한 후 연결 수정

삽입 위치까지 노드를 순서대로 탐색하므로 시간 복잡도는 $O(n)$ 이다.

7. delete(value)

`delete()` 는 입력한 값을 가진 첫 번째 노드를 삭제한다.

```
linked_list.delete("B")
```

삭제 전:

```
[A] <-> [B] <-> [C]
```

삭제 후:

```
[A] <-> [C]
```

가운데 노드를 삭제할 때는 이전 노드와 다음 노드를 서로 연결한다.

```
삭제 노드.prev.next = 삭제 노드.next
삭제 노드.next.prev = 삭제 노드.prev
```

삭제 위치에 따라 처리가 달라진다.

삭제 위치	처리
첫 번째 노드	<code>head</code> 를 다음 노드로 변경
마지막 노드	<code>tail</code> 을 이전 노드로 변경
중간 노드	이전 노드와 다음 노드를 서로 연결
값이 없음	<code>False</code> 반환

첫 번째 노드나 마지막 노드를 삭제한 뒤에는 새로운 `head.prev` 또는 `tail.next` 가 자연스럽게 `None` 이 되어야 한다.

값을 찾기 위해 리스트를 순회하므로 시간 복잡도는 $O(n)$ 이다.

8. find(value)

`find()` 는 입력한 값을 가진 첫 번째 노드의 인덱스를 반환한다.

```
linked_list.find("B")
```

리스트가 다음과 같다면:

```
인덱스:  0      1      2
         [A] <-> [B] <-> [C]
```

반환 결과는 `1` 이다.

찾는 값이 없으면 `-1` 을 반환한다.

```
index = linked_list.find("X")
print(index)  # -1
```

`head` 부터 노드를 하나씩 확인하므로 시간 복잡도는 $O(n)$ 이다.

9. get(index)

`get()` 은 입력한 인덱스에 있는 노드의 값을 반환한다.

```
value = linked_list.get(1)
print(value)
```

출력:

```
B
```

음수 인덱스이거나 존재하지 않는 인덱스라면 `None` 을 반환한다.

파이썬 리스트와 달리 링크드 리스트는 인덱스 위치로 바로 이동할 수 없다.

`head` 부터 원하는 위치까지 순서대로 이동해야 하므로 시간 복잡도는 $O(n)$ 이다.

10. get_node(index)

`get_node()` 는 특정 인덱스에 있는 `Node` 객체 자체를 반환한다.

```
node = linked_list.get_node(1)

print(node.value)
print(node.prev)
print(node.next)
```

`get()` 은 노드의 값만 반환하지만, `get_node()` 는 노드 전체를 반환한다는 차이가 있다.

유효하지 않은 인덱스이면 `None` 을 반환한다.

11. get_prev(index)

`get_prev()` 는 특정 인덱스 노드의 이전 노드를 반환한다.

```
[A] <-> [B] <-> [C]
      ↑
    index 1
```

`get_prev(1)` 은 값이 `A` 인 노드 객체를 반환한다.

첫 번째 노드는 이전 노드가 없으므로 `get_prev(0)` 의 결과는 `None` 이다.

12. get_next(index)

`get_next()` 는 특정 인덱스 노드의 다음 노드를 반환한다.

```
[A] <-> [B] <-> [C]
      ↑
    index 1
```

`get_next(1)` 은 값이 `C` 인 노드 객체를 반환한다.

마지막 노드는 다음 노드가 없으므로 결과는 `None` 이다.

13. length()

`length()` 는 현재 저장된 노드의 개수를 반환한다.

```
linked_list.append("A")
linked_list.append("B")
linked_list.append("C")

print(linked_list.length())
```

출력:

```
3
```

현재 구현은 별도의 길이 변수를 저장하지 않는다.

따라서 `head` 부터 마지막 노드까지 모두 순회하며 개수를 센다.

시간 복잡도는 $O(n)$ 이다.

14. display()

`display()` 는 `head` 부터 `tail` 까지 정방향으로 이동하며 리스트를 문자열로 만든다.

```
print(linked_list.display())
```

출력 예시:

```
HEAD <-> [A] <-> [B] <-> [C] <-> None
```

`next` 를 따라 모든 노드를 한 번씩 방문하므로 시간 복잡도는 $O(n)$ 이다.

15. display_reverse()

`display_reverse()` 는 `tail` 부터 `head` 까지 역방향으로 이동하며 리스트를 문자열로 만든다.

```
print(linked_list.display_reverse())
```

출력 예시:

```
TAIL <-> [C] <-> [B] <-> [A] <-> None
```

`prev` 를 따라 모든 노드를 방문한다.

단일 링크드 리스트와 달리 이전 노드 참조를 가지고 있기 때문에 역방향 순회가 가능하다.

16. 메서드 요약

메서드	역할	시간 복잡도
<code>append(value)</code>	맨 뒤에 노드 추가	$O(1)$
<code>prepend(value)</code>	맨 앞에 노드 추가	$O(1)$
<code>insert(index, value)</code>	특정 인덱스에 노드 삽입	$O(n)$
<code>delete(value)</code>	일치하는 첫 번째 노드 삭제	$O(n)$
<code>find(value)</code>	값이 있는 인덱스 검색	$O(n)$
<code>get(index)</code>	인덱스에 있는 값 조회	$O(n)$
<code>get_node(index)</code>	인덱스에 있는 노드 조회	$O(n)$
<code>get_prev(index)</code>	이전 노드 조회	$O(n)$
<code>get_next(index)</code>	다음 노드 조회	$O(n)$
<code>length()</code>	노드 개수 계산	$O(n)$
<code>display()</code>	정방향 구조 출력	$O(n)$
<code>display_reverse()</code>	역방향 구조 출력	$O(n)$

더블 링크드 리스트를 구현할 때 가장 중요한 부분은 노드를 삽입하거나 삭제할 때 `prev`와 `next`를 모두 올바르게 수정하는 것이다.